

AP1 : Web Component - Planning

Titre de la séquence :	AP	Auteur :	RINALDI Axel
Objectifs pédagogiques			
B2 compétences : - Participer à la conception de l'architecture d'une solution applicative - Exploiter les ressources du cadre applicatif (framework) - Exploiter les technologies Web pour mettre en œuvre les échanges entre applications, y compris de mobilité - Intégrer en continu les versions d'une solution applicative - Réaliser les tests nécessaires à la validation ou à la mise en production d'éléments adaptés ou développés Savoirs associés: - Méthodes, normes et standards associés au processus de conception et de développement d'une solution applicative - Architectures applicatives : concepts de base et typologies - Interfaces homme-machine : principes ergonomiques, techniques de conception, d'évaluation et de validation. - Concepts de la programmation événementielle : techniques de gestion des événements et exploitation de bibliothèques de composants graphiques - Caractéristiques des formats de données : structurées ou non - Fonctionnalités d'un outil de gestion de projets. - Fonctionnalités avancées d'un environnement de développement. - Techniques de gestion de versions			
Pré requis :	- Algorithmique, langage JAVA		
Activités :	- Programmation événementielle avec Java Script - Connaissance du DOM - Mise en œuvre de l'outil de gestion de version GIT		
Public :	BTS	Durée prévue :	2 à 4 heures

L'objet de cet atelier professionnel est la réalisation d'un Web Component affichant un planning calendaire avec la possibilité de créer, modifier et supprimer graphiquement des événements au sein d'une application WEB.

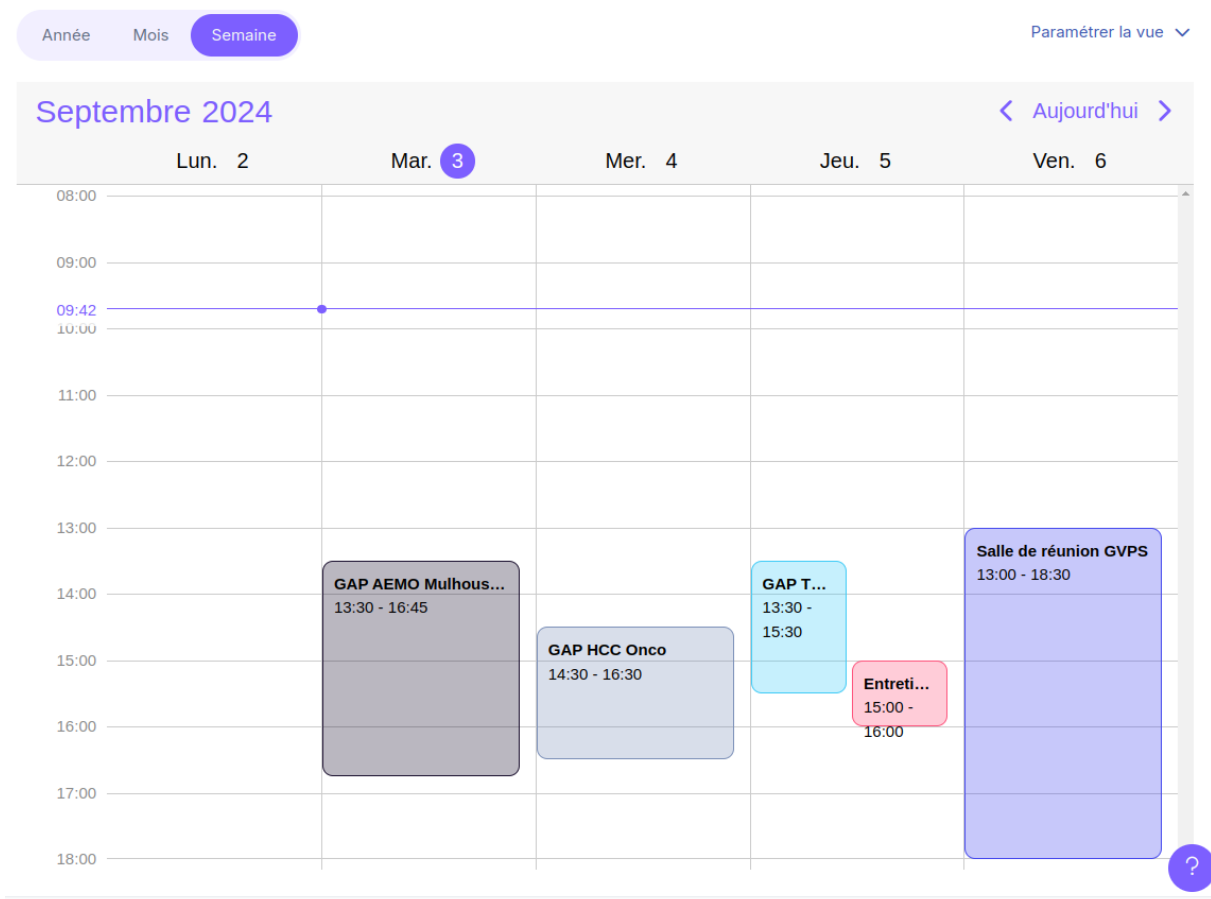
1. Présentation du projet et cahier des charges	3
1.1. Affichage et navigation	3
1.2. Manipulation des créneaux horaires associés aux événements.	4
1.3. Création d'un planning en JS	5
1.4. Gestion des paramètres du planning	6
1.5. Gestion de la persistance des données du planning	6
1.6. Quelques méthodes nécessaires à la gestion d'un créneau	7
1.7. Créer le Web Component « planning »	7
2. Documents attendus	8
2.1. Code fonctionnel complet du Web Component Planning	8
2.2. Documentation technique	8
2.3. Documentation utilisateur WEB	8
2.4. Journal de jour	8
2.5. Présentation de votre projet	9
3. Consignes générales	9
4. Proposition d'une démarche à suivre	9

1. Présentation du projet et cahier des charges

L'objectif de ce projet est la réalisation d'un Web Component (Composant WEB) affichant une vue calendrier d'un planning dont un exemple de représentation est donnée dans la figure ci-dessous. Le composant propose trois vues calendrier : Année, Mois et Semaine. Vous développerez en premier lieu la vue Semaine.

1.1. Affichage et navigation

1) La vue semaine affiche les jours en colonne et les heures en ligne. Un paramètre permet de visualiser uniquement les jours et heures ouvrés ou tous les jours d'une semaine et toutes les heures.



2) La navigation entre les semaines se fait avec les deux boutons « < » « > » et/ou par une barre de scrolling horizontale en bas du composant et/ou l'utilisation des touches de flèches gauche et droite du clavier.

3) La navigation entre les heures se fait via une barre de scrolling verticale à droite et/ou la roulette de la souris et/ou l'utilisation des touches de flèches haut et bas du clavier.

4) Dans tous les cas la barre horizontale d'entête affichant les noms et dates des jours doit rester visible ainsi que la colonne de gauche affichant les horaires.

5) Les événements planifiés apparaissent sous la forme de créneaux horaires, rectangles colorés aux coins arrondis et dont les dimensions et positions correspondent au jour et heures de début et de fin de l'événement. La durée d'un créneau horaire ne peut pas dépasser le cadre d'une journée. En revanche un événement peut s'étaler sur plusieurs jours et son affichage se fera via plusieurs créneaux horaires (un par jour).

6) Le planning doit permettre l'affichage de plusieurs créneaux horaires planifiés en même temps ou partageant des zones horaires communes. La largeur de ces derniers doit être adaptée en conséquence. (Cas du jeudi sur l'image ci-dessus).

7) Les dimensions de l'ensemble de la vue du planning peuvent être modifiées par l'utilisateur, le Web Component doit alors adapter son affichage en conséquence de manière à respecter les points décrits ci-dessus.

1.2. Manipulation des créneaux horaires associés aux événements.

8) Chaque événement est composé au minimum d'un identifiant numérique, d'une date et heure de début, d'une date et heure de fin, d'un libellé et d'une couleur. Vous créez la classe mère **Evenement** rassemblant ces informations. Pour l'ajout d'informations additionnelles, le programmeur utilisant votre Web Component créera une classe fille de la classe **Evenement**.

9) Un événement est affiché au planning sous la forme de un ou plusieurs créneaux horaires. Un créneau horaire est affiché sous la forme d'un rectangle coloré aux coins arrondis comme sur la figure précédente. Dans le cas où la durée d'un événement dépasse une journée celui-ci sera affiché à l'aide de plusieurs créneaux. Exemple : un événement commençant le 03/09/2024 à 15h00 et se terminant le 05/09/2024 à 10h00 sera affiché avec trois créneaux : le premier pour le 03/09/2024 de 15h00 à 24h00, le second pour le 04/09/2024 de 00h00 à 24h00 et le troisième pour le 05/09/2024 de 00h00 à 10h00.

Chaque créneau contient donc la date du jour, une heure de début, une heure de fin et une référence vers l'événement associé.

Vous créez la classe **Creneau** qui rassemblera ces données.

10) Pour créer un événement dont la durée dépasse le cadre d'une journée, l'utilisateur devra appuyer sur un bouton « **Nouveau** » qui fera apparaître un formulaire permettant de saisir le libellé, la date et heure de début, la date heure de fin et de sélectionner une couleur. Le formulaire créera une instance de la classe **Evenement** pour gérer le nouvel événement. La classe **Evenement** disposera de la méthode « **gereCreneaux** » qui permettra de créer automatiquement les créneaux sous la forme d'instance de la classe **Creneau**. Le même formulaire permettra aussi de modifier ou supprimer un créneau existant. Chaque instance de la classe

Evenement devra contenir l'attribut **creneaux**, tableau regroupant l'ensemble des créneaux associés à l'événement dont chaque case contiendra la référence d'un créneau.

11) Pour créer un événement dont la durée est inférieure à une journée l'utilisateur pour utiliser le formulaire décrit précédemment ou bien le faire graphiquement directement en créant un créneau avec la souris. Un double clic sur le créneau permettra alors d'afficher le formulaire de mise à jour de l'événement dont la date et de début ainsi que la date et heure de fin seront déjà renseignées.

12) Vous créerez la classe **Planning** qui regroupera l'ensemble des événements au travers de l'attribut **evenements**, tableau dont chaque case contiendra la référence d'un événement (instance de la classe **Evenement**).

13) Il doit être possible de créer ou modifier graphiquement un créneau directement sur le planning avec la souris. Pour cela, l'utilisateur clique sur le planning à l'endroit correspondant au jour et à l'heure de début, maintient le bouton gauche de la souris enfoncé et glisse cette dernière sur le planning jusqu'à l'heure de fin désirée et relâche le bouton gauche de la souris. La fin de cette opération entraîne automatiquement la création d'une instance d'événement et d'une instance de créneau associé et l'apparition du formulaire de mise à jour de l'événement.

14) Pour modifier un événement, il suffit de double cliquer sur l'un de ces créneaux pour faire apparaître le formulaire de mise à jour. Dans le cas d'un événement dont la durée est inférieure à la journée, un clic sur ce dernier doit faire apparaître des attributs graphiques permettant de déplacer ce dernier et/ou de modifier l'heure de début ou de fin avec la souris. Un appui sur la touche ECHAP permet de faire disparaître les attributs permettant de modifier le créneau avec la souris.

1.3. Création d'un planning en JS

15) Il doit être possible d'insérer votre Web Component directement dans du code JavaScript pour qu'il viennent s'insérer dans le contenu d'une balise HTML existante de la manière suivante :

Da
ns
la
par
tir
co
de
HT
ML
on
cré
e
un
e

```

<body>
  ...
  <div id="MonPlanning"></div>
  ...

  <script>
    let planning = new Planning() ;
    ...
    ...
    planning.creeObjetsHtml( document.getElementById( "MonPlanning" ) ) ;
  </script>
</body>

```

balise DIV destinée à contenir l'UI (Interface Utilisateur) du planning.

Dans la partie code JS on crée un objet de type **Planning** dont on stocke la référence dans une variable, ici la variable **planning**. Puis, après quelques initialisations diverses on demande au planning de créer dynamiquement les objets HTML constitutifs de son UI au sein de l'objet HTML DIV défini plus haut en appelant la méthode **createObjetHtml** que vous aurez implémenté dans la classe **Planning**.

1.4. Gestion des paramètres du planning

16) La classe Planning doit proposer la méthode **setParametres(Object)** permettant de définir les paramètres de fonctionnement du planning par exemple une de choix de couleur pour les créneaux ou un booléen permettant de préciser si les événements admettent de gérer un ou plusieurs créneaux, etc.
La méthode **setParametre** prend en argument un objet défini au format JSON. Vous trouverez ci dessous un exemple d'utilisation qui peut être complété.

```
let parametres = {
  creneau: {
    choixCouleurs: ["#0000FF", "#FF0000", "#0000FF"], // Liste de choix de couleur
    factory: ()=>{ return new Creneau(); } // Fonction de création d'un créneau
  },
  evenement: {
    multiCreneaux: true, // Multi-créneaux,
    factory: ()=>{ return new Evenement(); } // Fonction de création d'un événement
  }
};

planning.setParametres( parametres );
```

Le champ « **factory** » défini dans les deux objets référencés respectivement par les attributs « **creneau** » et « **evenement** » contiennent chacun une fonction JavaScript qui quand on les appelle créent respectivement un objet de type **Creneau** ou de type **Evenement**. Ces fonctions sont appelées « **fonction usine (factory)** », elles servent à créer un objet à partir d'une classe. Leur intérêt, dans ce cas, est de permettre à un programmeur utilisant votre Web Component de personnaliser ce dernier au niveau des créneaux et des événements. En effet il suffit de créer une classe personnalisée de créneau fille de la classe **Creneau**, par exemple **CreneauPersonnalise** et de modifier la fonction usine pour qu'elle crée un objet du type de la nouvelle classe.

17) La classe Planning doit proposer la méthode **getParametres()** qui renvoie un objet au format JSON avec les paramètres utilisés par la planning.

18) Pour permettre la persistance des paramètres, ces derniers peuvent être enregistrés directement dans le **Local Storage** du navigateur à l'aide d'une clé de type chaîne de caractères. Pour cela, la classe **Planning** doit proposer les méthodes **loadParametresFromLocalStorage(clé)** et **saveParametresToLocalStorage(clé)**.

La figure ci-dessous donne un exemple d'utilisation de la première méthode permettant de charger les paramètres du planning stockés dans le Local Storage du navigateur référencés avec la clé « Planning-Parametres »

```
planning.loadParametresFromLocalStorage( "Planning-Parametres" ) ;
```

1.5. Gestion de la persistance des données du planning

Les données affichées et mises à jour par le planning doivent être stockées sur un support. Ce dernier peut être sous la forme d'un objet JSON contenant un tableau de tous les événements. Chaque événement contient alors un tableau de tous les créneaux qui lui sont associés. Les mêmes données peuvent aussi être contenues dans des tables d'une base de données relationnelles (SqlServer, MySql ou MariaDB).

19) Le planning doit pouvoir accéder à ces données en lecture à partir d'une requête du type :

- « Donne moi l'ensemble des événements et créneaux pour la période comprise entre une date de début et une date de fin »

Il doit pouvoir aussi accéder en écriture aux données à partir de requêtes du type :

- « Enregistre les données de l'événement numéro XXXX avec ces données (objet contenant les données). »

- « Enregistre les données du créneau du jour J appartenant à l'événement numero XXXX avec ces données (objet contenant les données). »

- « Supprime l'événement numéro XXXX de la base de données »

- « Supprime le créneau du jour J appartenant à l'événement numéro XXXX de la base de données.»

Il faudra imaginer une manière de faire qui fonctionne dans un premier temps avec objet JSON et qui puisse être personnalisée pour des bases de données SQL.

Dans un premier temps vous concevrez une version utilisant un objet JSON pour stocker les données.

1.6. Quelques méthodes nécessaires à la gestion d'un créneau

20) La classe Creneau devra proposer au moins les méthodes suivantes :

- **creeObjetHTML(contener)** : Cette méthode génère les objets HTML pour afficher le créneau associé dans l'espace correspondant à une journée. La référence de cet espace (objet HTML) est donnée dans l'argument **contener**. Cette méthode sera appelée par la méthode du même nom présente dans la classe Evenement au moment de l'affichage du planning.

- **creeObjetHTMLPourMiseAJour(contener)** : Cette méthode sera appelée quand on clique sur un créneau. Elle devra créer les objets HTML permettant de modifier avec la souris le créneau.

1.7. Créer le Web Component « planning »

21) Il doit être possible d'insérer un planning dans une page WEB par l'utilisation d'un Web Component qui permet de définir la nouvelle balise HTML « planning » comme dans l'exemple ci-dessous :

```
<body>
|
|  ...
|  <planning></planning>
|  ...
|
```

La création d'un Web Component consiste en fait à créer un nouveau type de balise. Il est bien évidemment possible d'y associer des propriétés.

2. Documents attendus

2.1. Code fonctionnel complet du Web Component Planning

Le code sera délivré sous la forme d'un accès à la branche « master » d'un dépôt GIT.

Ce dépôt comportera la branche supplémentaire « test_web_component » montrant un cas d'utilisation complet du Web Component. Dans tous les cas l'ensemble du code source du Web Component et des tests doit être commenté.

2.2. Documentation technique

Cette documentation doit présenter le diagramme UML de classe complet ou par partie des éléments constituant le Web Component.

Chaque classe et chaque méthode doit faire l'objet d'une description permettant à un autre technicien en informatique de reprendre et continuer le développement du Web Component.

Documentation programmeur

Cette documentation doit permettre à un programmeur WEB d'utiliser et de personnaliser votre Web Component.

2.3. Documentation utilisateur WEB

Cette documentation doit permettre à un utilisateur WEB (non informaticien) d'utiliser votre Web Component Planning. Par exemple : Comment créer un événement et planifier des créneaux, comment changer la couleur associée à une événement, ...

2.4. Journal de jour

Vous tiendrez un journal de jour sous la forme d'un document « journal.txt » (pas Word ou Writer) dans lequel vous consignerez quotidiennement toutes les observations, problèmes rencontrés, bug et correction de bug. Ce fichier sera mis à jour avec Visual Studio Code et fera partie du dépôt GIT.

2.5. Présentation de votre projet

En fin de réalisation du projet vous ferez une présentation avec démonstration de ce dernier de 15 minutes devant vos camarades et moi même. Vous réaliserez un petit document Power point ou équivalent comme support de votre présentation.

3. Consignes générales

- Ce projet est réalisé en équipe de deux étudiants SLAM.
- Vous mettrez en place un dépôt GIT de référence pour gérer le projet et ses mises à jour. Je vous conseille de créer un compte sur Github et d'y créer votre dépôt dessus de manière à pouvoir y accéder de n'importe où.
- L'ensemble du code source doit être de votre propre création et l'objet d'une copie.
- Vous tiendrez à jour votre fichier **journal.txt** avec la répartition des tâches.
- Le délais de réalisation et de rendu est fixé à la **fin des vacances de la Toussaint**.

4. Proposition d'une démarche à suivre

Il est clair qu'à ce jour vous ne disposez pas encore de toutes les compétences pour réaliser le projet. Les enseignements à venir vont vous permettront d'avancer sur le projet mais il vous faudra aussi chercher par vous même sur Internet. Cela fait partie du travail de développeur.

Je vous suggère de commencer par créer en HTML, CSS l'interface graphique du planning pour appréhendez ce que vous devrez coder en JavaScript.

L'un de vous peut aussi étudier comment faire un Web Component et faire des tests : https://developer.mozilla.org/en-US/docs/Web/API/Web_components

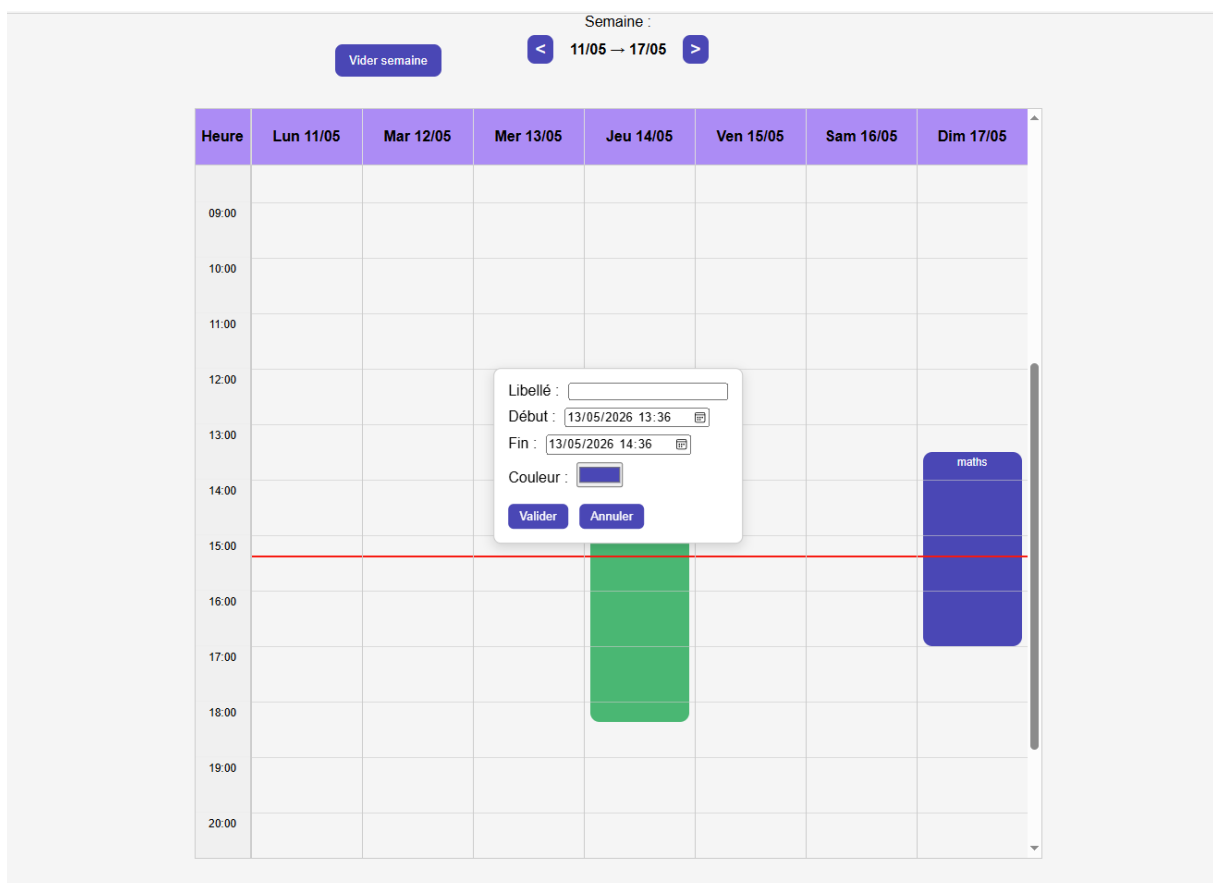
Vous pouvez faire de même pour l'accès au Local Storage : <https://fr.javascript.info/localstorage>

Vous aurez aussi besoin de gérer des traitements asynchrones avec l'aide de « promise » et des instructions « async » et « await » : https://www.pierre-giraud.com/javascript-apprendre-coder-cours/async-await/#google_vignette

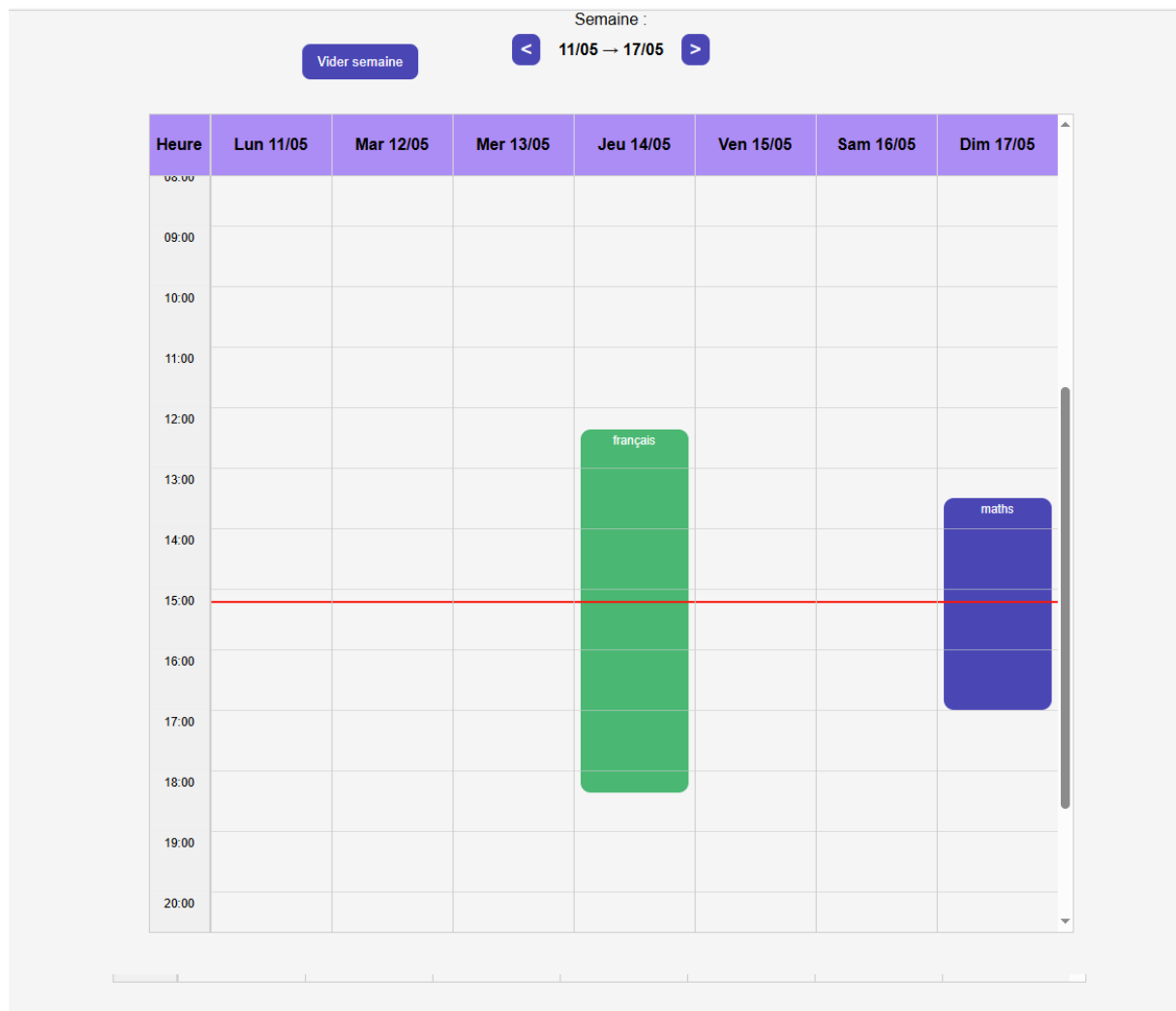
Réalisez un modèle HTML – CSS du formulaire de création et mise à jour d'un événement.

Réfléchissez à comment vous allez permettre de modifier graphiquement avec la souris les données d'un créneau. Regardez comment gérer des événements logiciels liés à la souris : https://www.w3schools.com/js/js_html_dom_eventlistener.asp

1. Réalisation



Barre horizontale pou



Possibilité de modifier et de supprimer des évènements.

